

第七章 图

第一讲

7.1 图的定义与术语

7.2 图的存储结构

练习与作业

第二讲

7.3 图的遍历

7.4 图的连通性问题

练习与作业

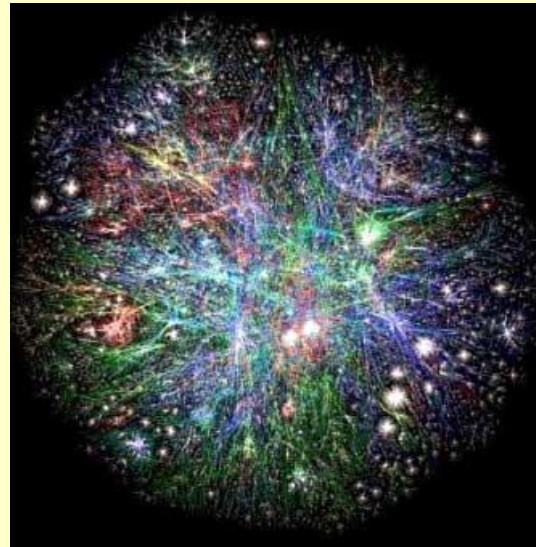
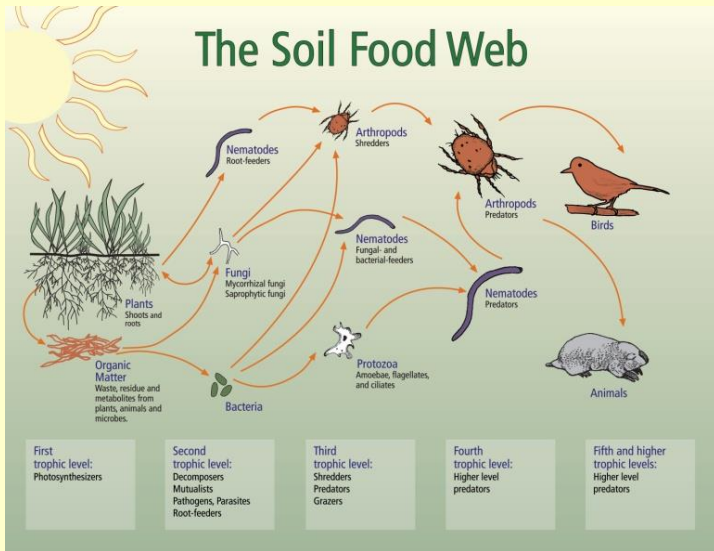
第三讲

7.5 有向无环图及其应用

7.6 最短路径

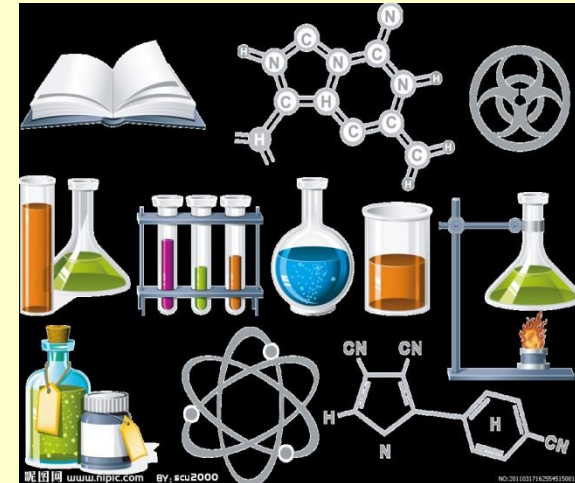
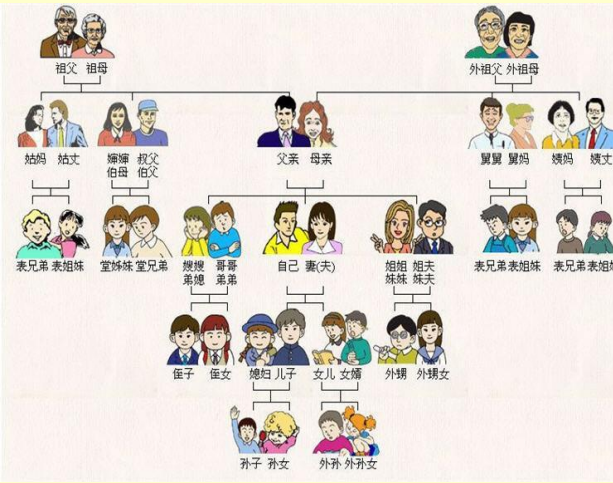
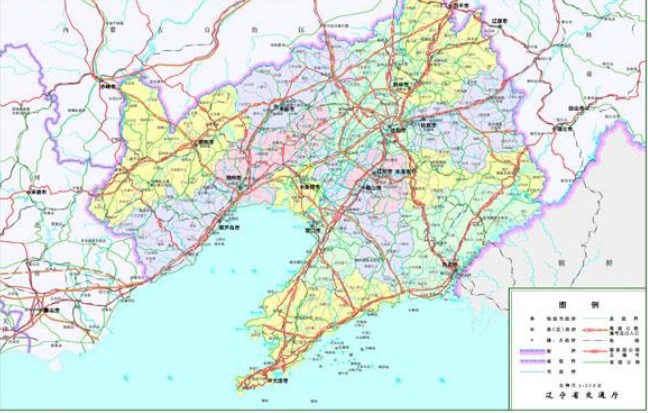
练习与作业

The Soil Food Web



Graphs are **everywhere**, and quite a few are **huge** graphs!

辽宁省公路网图



一、图的基本定义和术语

图(Graph)——非空的顶点集合和边（或者弧）的集合组成

$$G = (V, \{E\})$$

$$V = \{v_i \mid v_i \in \text{DataObject}\}$$

$$E = \{ \langle v_i, v_j \rangle \mid v_i, v_j \in V \wedge P(v_i, v_j) \}$$

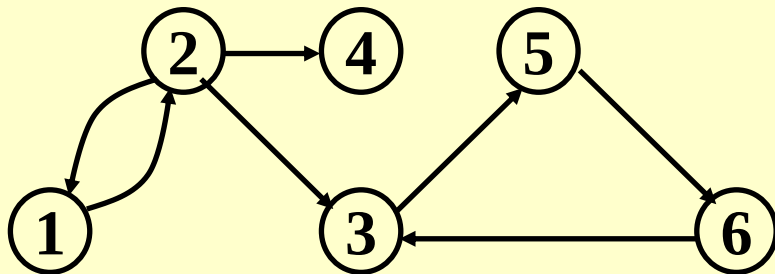
V: 图G中顶点的非空有限集;

E: 图G中边的有限集合，边是顶点的**无序对**或**有序对**。

❖ **有向图**：如果任意两个顶点构成的边 $\langle v_i, v_j \rangle \in E$ 是有序的，即顶点之间的连线是有方向的，图为有向图，称 v_i 为**弧尾或初始点**， v_j 为**弧头或终端点**。

❖ **无向图**：如果任意两个顶点构成的边 $(v_i, v_j) \in E$ 是无序的，即顶点之间的连线是没有方向的，图为无向图。顶点的无序对，记为 (v_i, v_j) 或 (v_j, v_i) ，并且 $(v_i, v_j) = (v_j, v_i)$

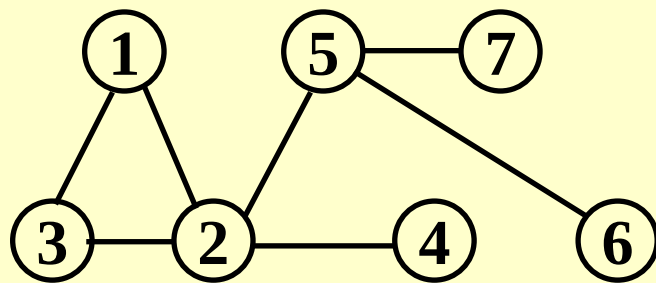
➤ 有向图vs.无向图



有向图 G_1

$G_1=(V_1,\{E_1\})$ 中: $V_1=\{1,2,3,4,5,6\}$

$E_1=\{<1,2>, <2,1>, <2,3>, <2,4>, <3,5>, <5,6>, <6,3>\}$



无向图 G_2

$G_2=(V_2,\{E_2\})$: $V_2=\{1,2,3,4,5,6,7\}$

$E_2=\{(1,2), (1,3), (2,3), (2,4), (2,5), (5,6), (5,7)\}$

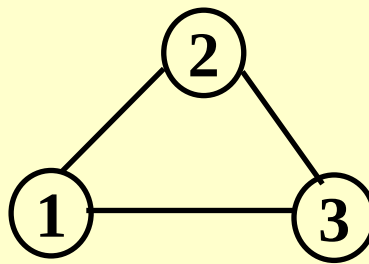
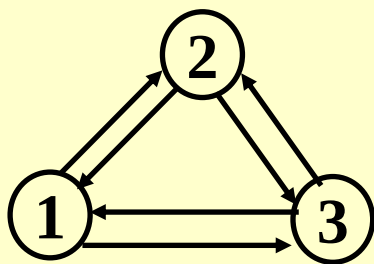
➤ 稀疏图vs.稠密图

n 表示图中顶点的数目

e 表示边或弧的数目

不考虑顶点到自身的弧或边:

- ◆ 无向图中 e 取值范围0到 $n(n-1)/2$, 将有 $n(n-1)/2$ 条边的无向图成为**无向完全图**;
- ◆ 有向图中 e 的取值范围是0到 $n(n-1)$, 将具有 $n(n-1)$ 条弧的有向图称为**有向完全图**。

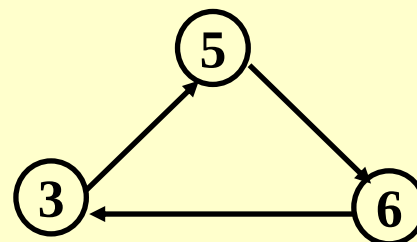
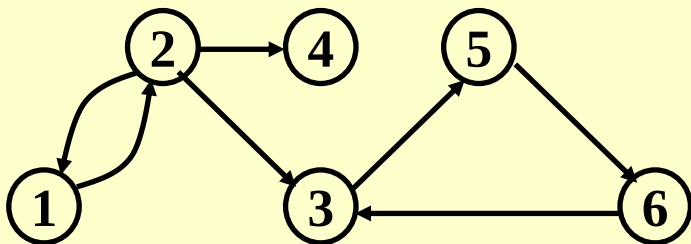


稀疏图: 边或弧较少的图 ($e < n \log n$)

稠密图: 边或弧较多的图 ($e \geq n \log n$)

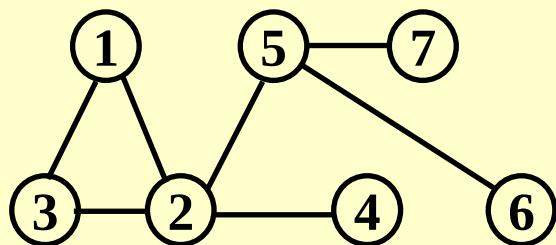
➤ 图vs.网

- ◆ **权**:与图的边或弧相关的数叫**权**;
- ◆ **网**:带权的图叫**网**;
- ◆ **子图**:如果图 $G(V, \{E\})$ 和图 $G'(V', \{E'\})$,满足: $V' \subseteq V$ 且 $E' \subseteq E$, 则称 G' 为 G 的子图;



图与子图

- ◆ **邻接点**: 对于无向图 $G=(V, \{E\})$, 如果存在边 (v, v') 则称 v 和 v' 互为**邻接点**。边 (v, v') 依附于顶点 v 、 v' , 边 (v, v') 与顶点 v 、 v' **相关**。顶点 v 的**度**为与每个顶点相连的边数, 记为 $TD(v)$ 。



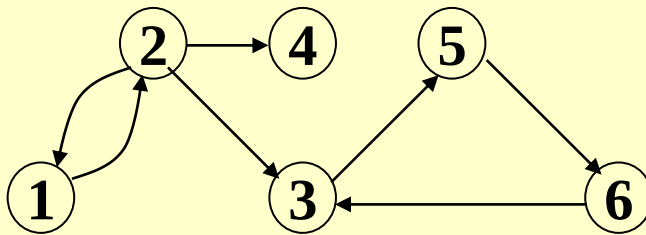
G_2

顶点5的度: 3

顶点2的度: 4

- ◆ 对于有向图，如果存在边 $\langle v, v' \rangle$ 则称顶点 v 邻接到顶点 v' ，或者顶点 v' 邻接自顶点 v 。顶点的度分成入度与出度， $TD(v) = ID(v) + OD(v)$ ，其中：

- 入度：以该顶点为头的弧的数目，记为 $ID(v)$
- 出度：以该顶点为尾的弧的数目，记为 $OD(v)$



G1

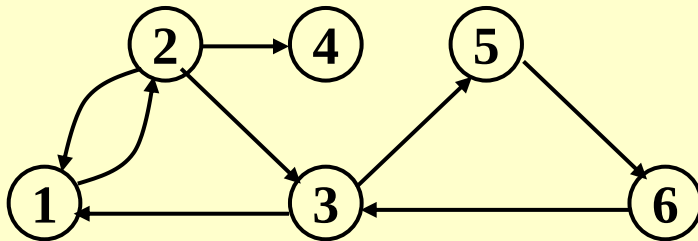
顶点2入度：1 出度：3
顶点4入度：1 出度：0

- ◆ 对于具有 n 个顶点、 e 条边的图，顶点 v_i 的度 $TD(v_i)$ 与顶点的个数以及边的数目满足关系：

$$e = \frac{1}{2} \sum_{i=1}^n TD(v_i)$$

➤ 路径

- **路径**: 路径是顶点的序列 $V=\{V_{i0}, V_{i1}, \dots, V_{in}\}$, 满足 $(V_{i,j-1}, V_{ij}) \in E$ 或 $\langle V_{i,j-1}, V_{ij} \rangle \in E, (1 \leq j \leq n)$
- **路径长度**: 沿路径边的数目或沿路径各边权值之和。
- **回路 (环)**: 第一个顶点和最后一个顶点相同的路径。
- **简单路径**: 序列中顶点不重复出现的路径。
- **简单回路 (环)**: 除了第一个顶点和最后一个顶点外, 其余顶点不重复出现的回路。



G_1

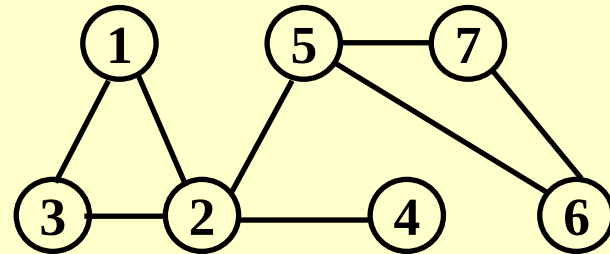
路径: 1, 2, 3, 5, 6, 3

路径长度: 5

简单路径: 1, 2, 3, 5

回路: 1, 2, 3, 5, 6, 3, 1

简单回路: 3, 5, 6, 3



G_2

路径: 1, 2, 5, 7, 6, 5, 2, 3

路径长度: 7

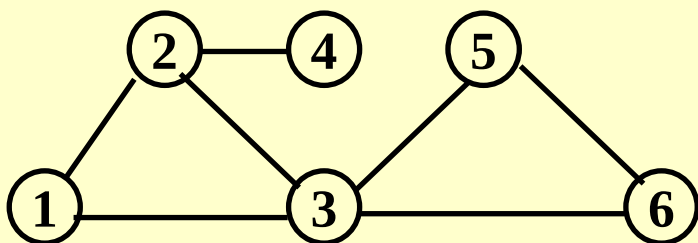
简单路径: 1, 2, 5, 7, 6

回路: 1, 2, 5, 7, 6, 5, 2, 1

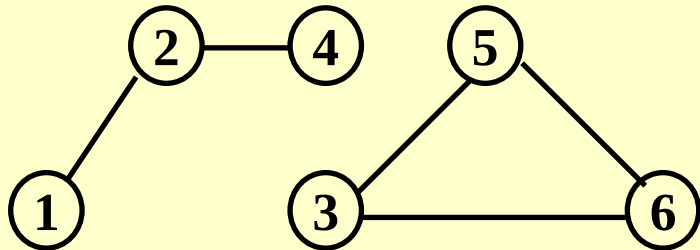
简单回路: 1, 2, 3, 1

➤ 连通性

- **连通**：在无向图中，从顶点 V 到顶点 W 有一条路径，则说 V 和 W 是连通的
- **连通图**：在无向图中，图中任意两个顶点都是连通的叫连通图
- **连通分量**：非连通图中的极大连通子图叫连通分量。

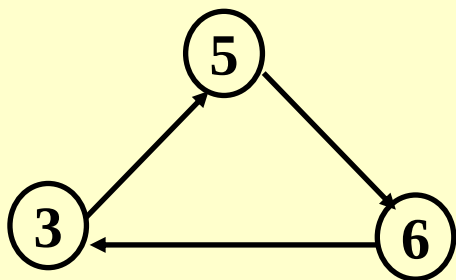


连通图



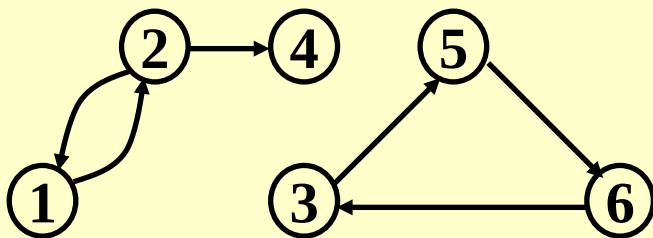
非连通图
连通分量

- **强连通图**：有向图中，如果对每一对 $V_i, V_j \in V, V_i \neq V_j$ ，从 V_i 到 V_j 和从 V_j 到 V_i 都存在路径，则称 G 是强连通图。
- **强连通分量**：非强连通图中的极大强连通子图称作该有向图的强连通分量



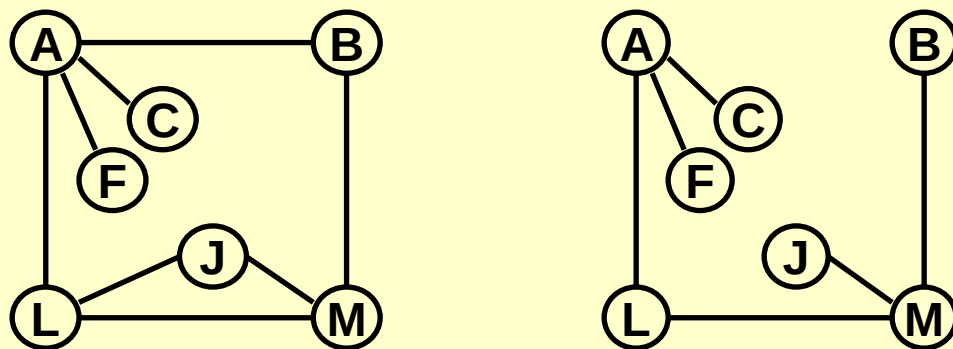
强连通图

定理：一个有向图是强连通的，当且仅当 G 中有一个回路，它至少包含每个节点一次。



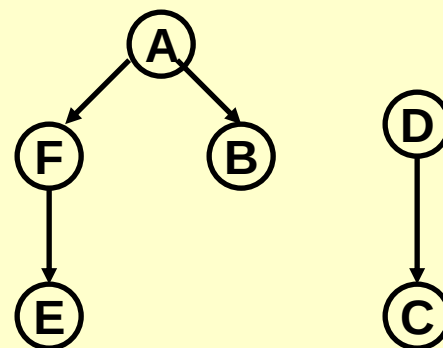
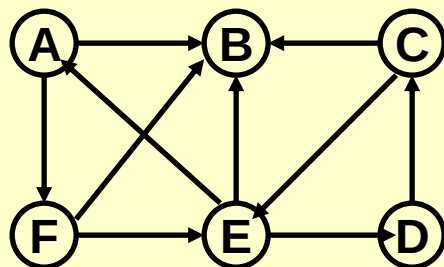
非强连通图与连通分量

- **生成树**：是连通图G的极小连通子图，它含有其全部 n 个顶点，包含且仅包含G的 $n-1$ 条边。



特点：在生成树中添加任意一条属于原图中的边必定会产生回路，因为新添加的边使其所依附的两个顶点之间有了第二条路径。若生成树中减少任意一条边，则必然成为非连通的。

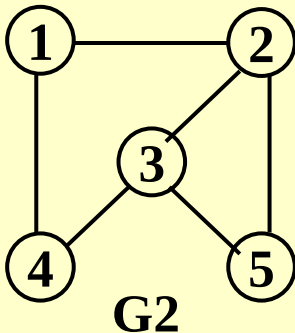
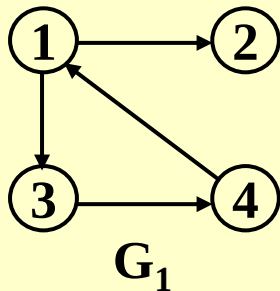
- **有向树**：如果一个有向图恰好有一个顶点的入度为0，其余顶点的入度均为1，则称为有向树。
- **生成森林**：由若干有向树组成，含有图中全部顶点，但只有足以构成若干棵不相交的有向树的弧。



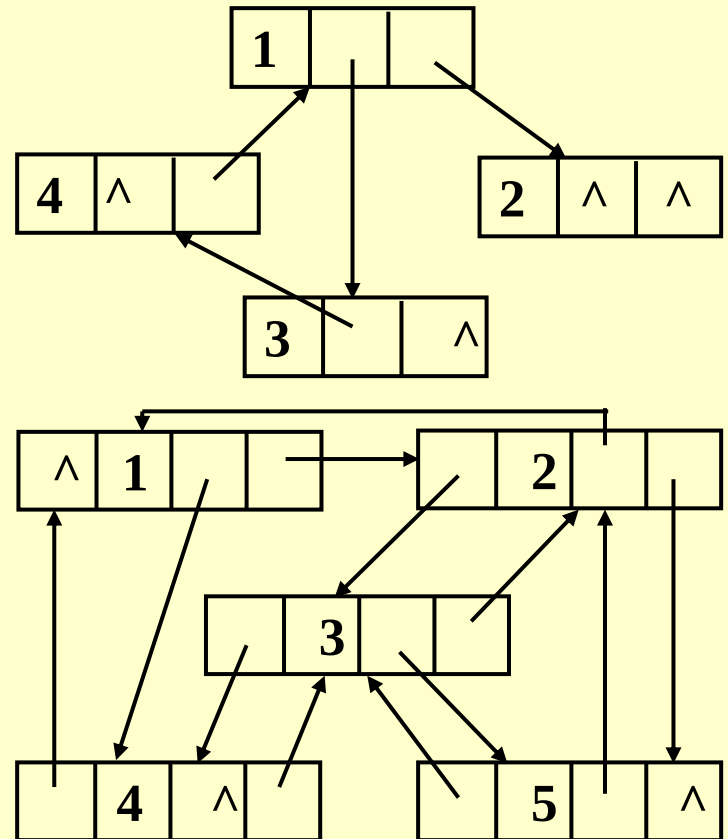
返回

数据结构存储的方式：

- 顺序映象：图的结构比较复杂，无法以数据元素在存储区中的物理位置表示元素之间的关系，即**图没有顺序存储结构，但可以借助数组的数据类型表示元素的关系。**
- 链式映象：采用多重链表表示图，用由一个数据域和多个指针域组成的节点表示图中的顶点。



存在问题：节点度数不一致，指针域数目设置困难。因此，实际中也不采用这结构，而是根据具体需要设置结点结构和表结构



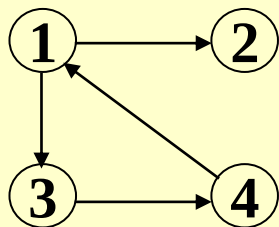
◆ 数组表示法

使用**邻接矩阵**表示顶点间相联关系。

定义：设 $G=(V,\{E\})$ 是有 $n \geq 1$ 个顶点的图， G 的邻接矩阵 A 是具有以下性质的 n 阶方阵：

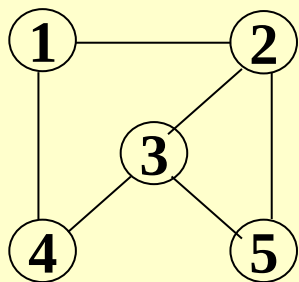
$$A[i][j] = \begin{cases} 1 & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{是} E(G) \text{中的边} \\ 0 & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{不是} E(G) \text{中的边} \end{cases}$$

G1



$$\begin{matrix} & \textcircled{1} & \textcircled{2} & \textcircled{3} & \textcircled{4} \\ \textcircled{1} & \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix} \\ \textcircled{2} & \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \\ \textcircled{3} & \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \\ \textcircled{4} & \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

G2



$$\begin{matrix} & \textcircled{1} & \textcircled{2} & \textcircled{3} & \textcircled{4} & \textcircled{5} \\ \textcircled{1} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \end{bmatrix} \\ \textcircled{2} & \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \end{bmatrix} \\ \textcircled{3} & \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \end{bmatrix} \\ \textcircled{4} & \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \end{bmatrix} \\ \textcircled{5} & \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

➤ 数组表示法，用两个数组分别存储：
数据元素（顶点的信息）和数据元素之间的关系（边或弧）的信息。

➤ 形式描述：

```
#define MAX_VERTEX_NUM 20      //最大顶点数设为20
```

```
typedef enum{dg,dn,udg,udn} GraphKind; //有向图,有向网,无向图,无向网
```

```
typedef struct ArcCell{
```

```
    VRType adj;          // 顶点关系类型
```

```
    InfoType *info;      //弧相关指针信息
```

```
}ArcCell,AdjMatrix[MAX_VERTEX_NUM][MAX_VERTEX_NUM]
```

```
typedef struct {
```

```
    VertexType vexs[MAX_VERTEX_NUM]; //顶点向量
```

```
    AdjMatrix arcs;                  //邻接矩阵表示弧
```

```
    int vexnum,arcnum;               //顶点数和边数
```

```
    GraphKind kind;
```

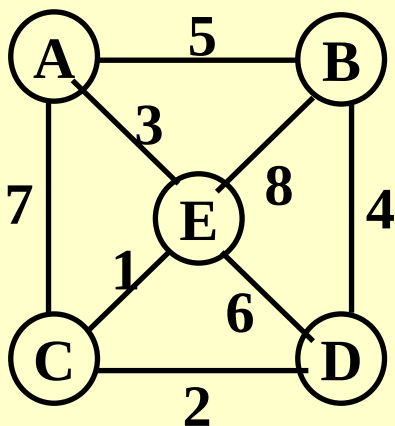
```
}Mgraph;
```

特点:

- 无向图的邻接矩阵对称, 可压缩存储; 有 n 个顶点的无向图需存储空间为 $n(n+1)/2$ 。
- 有向图邻接矩阵不一定对称; 有 n 个顶点的有向图需存储空间为 n^2 。
- 无向图中顶点 V_i 的度 $TD(V_i)$ 是邻接矩阵 A 中第 i 行元素之和。
- 有向图中,
 - 顶点 V_i 的___是 A 中第 i 行元素之和。(出度,入度)
 - 顶点 V_i 的___是 A 中第 i 列元素之和。(出度,入度)

➤ **网络的邻接矩阵可定义为: (带权的图)**

$$A[i][j] = \begin{cases} w_{ij} & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{是} E(G) \text{中的边} \\ \infty & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{不是} E(G) \text{中的边} \end{cases}$$



无向网 G_1

$G_1.arcs =$

	①	②	③	④	⑤
①	∞	5	7	∞	3
②	5	∞	∞	4	8
③	7	∞	∞	2	1
④	∞	4	2	∞	6
⑤	3	8	1	6	∞

$G_1.vexs = [A, B, C, D, E]$

$G_1.vexnum = 5$

$G_1.arcnum = 8$

$G_1.kind = UDN$

构造过程:

I顶点和边的数量

★ G.vexnum=5

★ G.arcnum=8

II顶点信息(数组存储A,B,C,D,E)

III边的信息(注意边权的初始化)

★ (A,B,5)

★ (A,E,3)

★ (A,C,7)

★ (B,E,8)

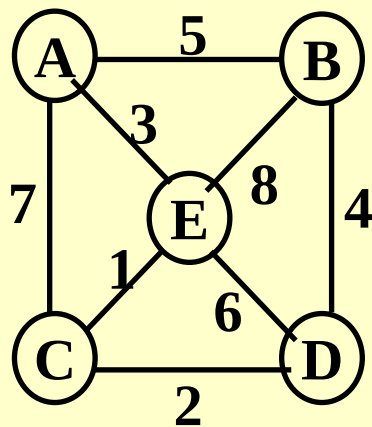
★ (C,E,1)

★ (B,E,4)

★ (D,E,6)

★ (C,D,2)

```
int locate(graph g, DataType
data)
{
    int i;
    for(i=0;i<g.vexnum;i++)
        if(g.vexs[i] == data)
            return i;
    return -1;
}
```



```
graph create_graph()
```

```
{
    graph g; int i,j,w, s1,s2;
    printf("Enter %d vertex/n",
g.vexnum);
    for(i=0;i<g.vexnum;i++) /* 输入顶点*/
    {
        printf("vex %d: ", i);
        scanf(" %c", &g.vexs[i]);
    }
    for(i=0;i<g.vexnum;i++) /* 初始化边权*/
        for(j=0;j<g.vexnum;j++)
            _____=INFINITY;
    printf("Enter %d arcs:(e.g. A B 5)/n",
g.arcnum);
    for(i=0;i<g.arcnum;i++) /* 输入边*/
    {
        printf("arc %d: ", i);
        scanf("%c %c %d", &ch1, &ch2, &w);
        tmp=getchar();
        s1=locate(_____); s2=locate(_____);
        _____=_____ =w; /* 边赋权值*/
    }

    return g;
}
```

◆ **邻接表**：为图中每个顶点建立一个单链表，第i个单链表中的结点表示依附于顶点 V_i 的边（有向图中 v_i 为弧尾）

头结点：typedef struct VNode{

VertexType data; //存放顶点信息

ArcNode *firstarc; //指示第一个邻接点

}VNode, **AdjList**[Max_VerTEX_Num];

data	firstarc
------	----------

表结点：typedef struct ArcNode { //

int adjvex; //存放与 V_i 邻接的点在表头数组中的位置

struct ArcNode *nextarc; //链域，指示下一条边或弧

InfoType *info;

}**ArcNode**;

adjvex	next	info
--------	------	------

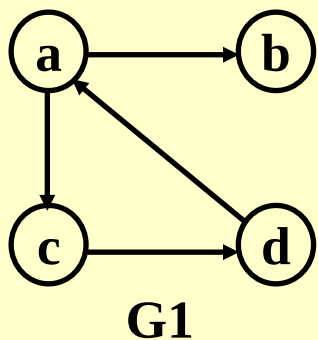
表：typedef struct {

AdjList vertices;

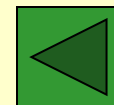
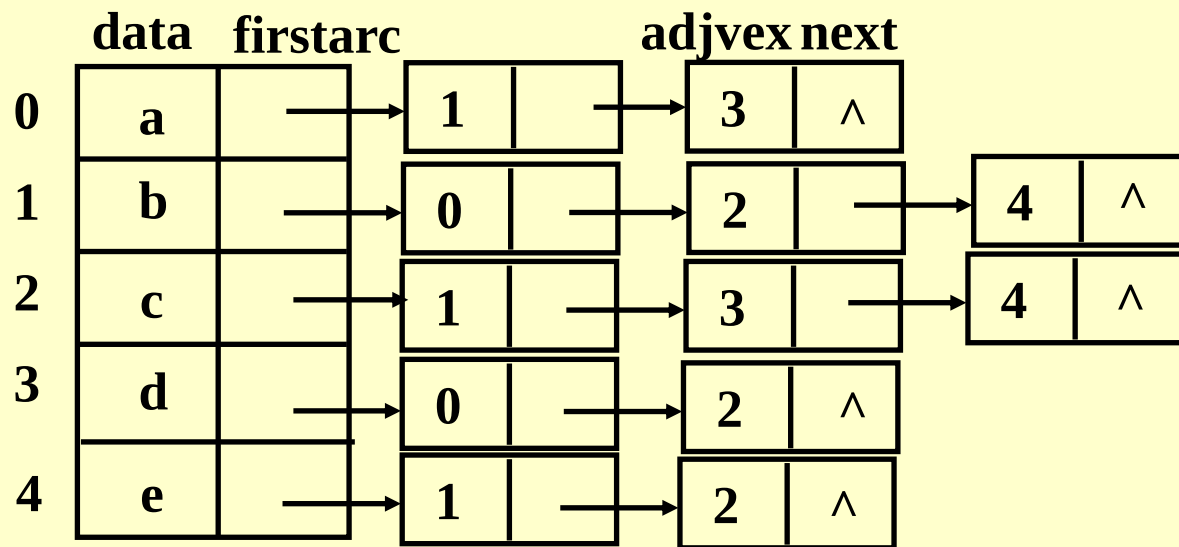
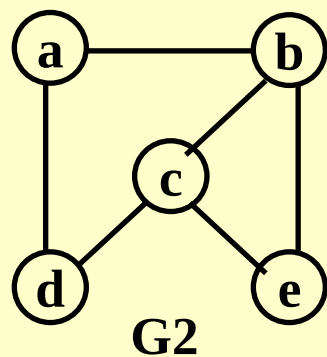
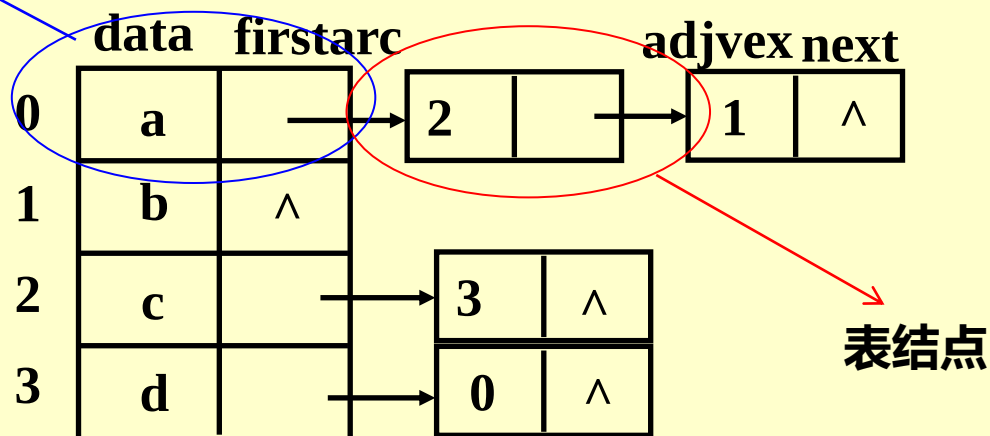
int vexnum, arcnum;

int kind;

} **AlGraph**

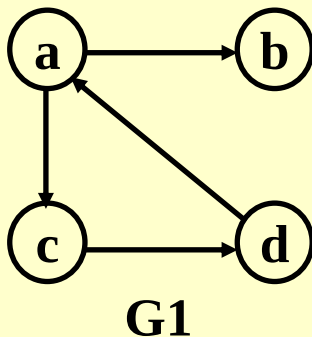


头结点



特点

- 无向图中顶点 V_i 的度为第 i 个单链表中的结点数;
- 有向图中
 - 顶点 V_i 的____为第 i 个单链表中的结点数;(出度, 入度)
 - 顶点 V_i 的____为整个单链表中邻接点域值是 i 的结点数
- 易找任一顶点的第一个邻接点和下一个邻接点, 判定任意两个顶点是否有边弧相连, 不及邻接矩阵。
- 逆邻接表: 有向图中对每个结点建立以 V_i 为头的弧的单链表。



	data	firstarc	adjvex	next
0	a	→	3	^
1	b	→	0	^
2	c	→	0	^
3	d	→	2	^

```
#define MAX_NUM 20//最大顶点数目
typedef enum {DG,DN,UDG,UDN} GraphKind;//图类型

typedef struct Acnode{
    int adjvex;//存放顶点在数组中的位置
    int value;//权值， 定义为整型
    struct Acnode *next;
    char *info;//存放弧的信息
}Acnode;//弧的定义
typedef struct {
    int data;
    Acnode *firstarc;
}VertexNode;//顶点定义
typedef struct{
    VertexNode Vertex[MAX_NUM];
    int Vexnum,Arcnum;//当前顶点数目、 弧数目
    GraphKind kind;//图的类型
}ALGraph;//图的定义
```

```
ALGraph G;
```

◆ 有向图的十字链表表示

- 顶点结点:

```
typedef struct VexNode{  
    VertexType data; //存与顶点有关信息  
    ArcBox *firstin; //指向以该顶点为弧头的第一个弧结点(向内)  
    ArcBox *firstout; //指向以该顶点为弧尾的第一个弧结点(向外)  
}VerNode;
```

data	firstin	firstout
------	---------	----------

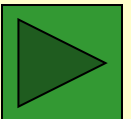
- 弧结点:

```
typedef struct ArcBox{  
    int tailvex, headvex; //弧尾、弧头在表头数组中位置  
    struct ArcBox *hlink; //指向弧头相同的下一条弧  
    struct arcbox *tlink; //指向弧尾相同的下一条弧  
    InfoType *info;  
}ArcBox;
```

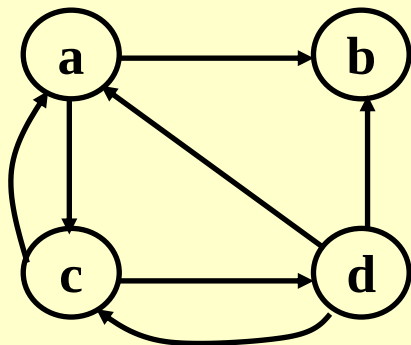
tailvex	headvex	hlink	tlink	info
---------	---------	-------	-------	------

- 图:

```
#define MAX_VERTEX_NUM 20  
typedef struct {  
    VerNode xlist[MAX_VERTEX_NUM]; //顶点数组  
    int vernum, arcnum;  
}OLGraph;
```



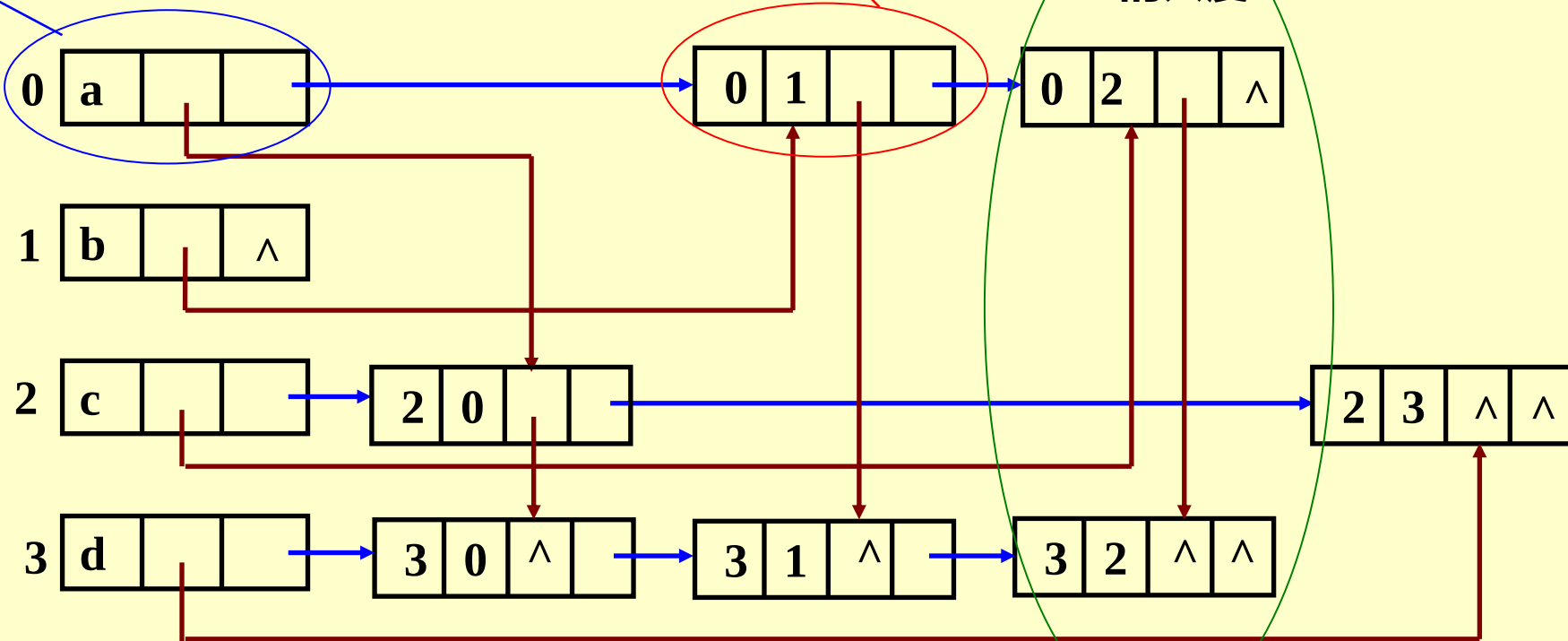
例



顶点结点

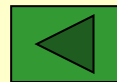
弧结点

c的入度



边用顶点位置表示

**十字链表表示法：结合了邻接表与逆邻接表
易求一个结点的入度与出度**



— 无向图的邻接多重表表示法

◆ 顶点结点:

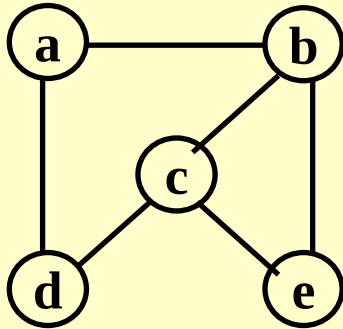
```
typedef struct VexBox{  
    VertexType data; //存与顶点有关的信息  
    EBox *firstedge; //指向第一条依附于该顶点的边  
}VerBox;
```

data	firstedge
------	-----------

◆ 边结点:

```
typedef enum {unvisited,visited} VisitIf;  
typedef struct EBox{  
    VisitIf mark; //标志域—访问标志  
    int ivex, jvex;  
    //该边依附的两个顶点在表头数组中位置  
    struct EBox *ilink, *jlink;  
    //分别指向依附于ivex和jvex的下一条边  
    InfoType *info;  
}EBox;
```

mark	ivex	ilink	jvex	jlink	info
------	------	-------	------	-------	------



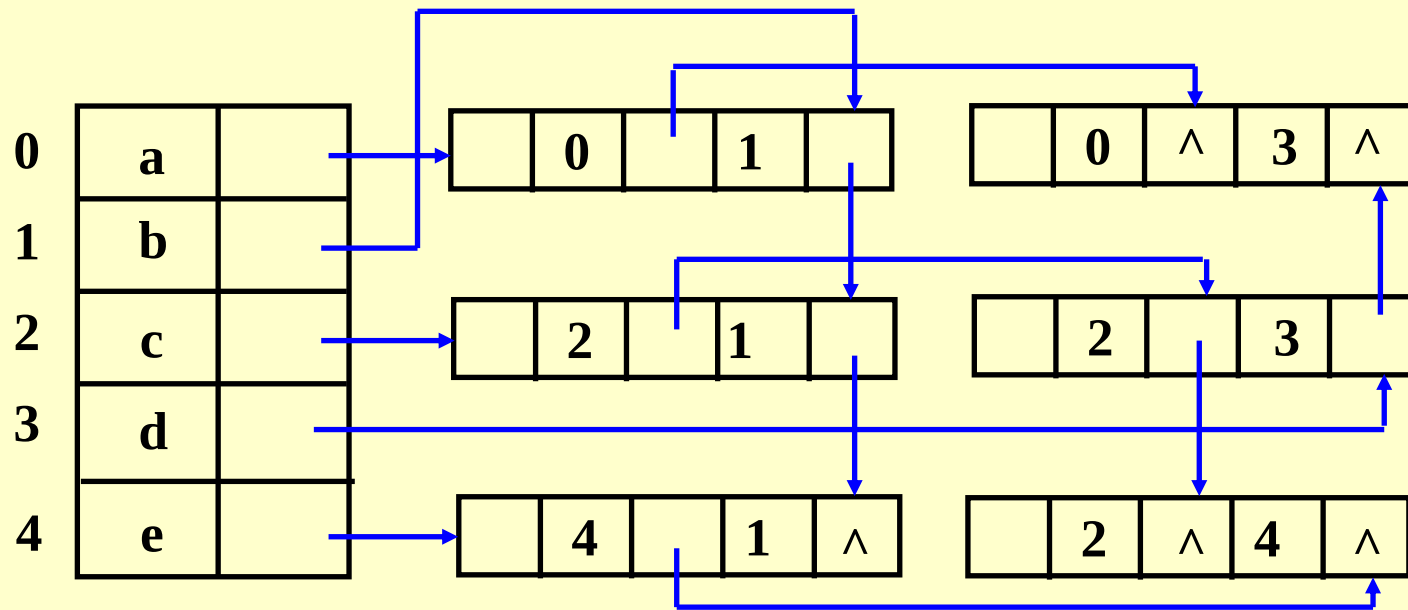
```
#define MAX_VERTEX_NUM 20
```

```
typedef struct {
```

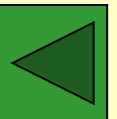
```
    VerBox adjmulist [MAX_VERTEX_NUM];
```

```
    int vernum,edgenum;
```

```
}AMLGraph;
```

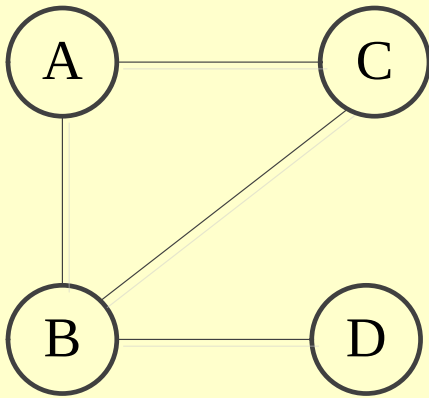


结果唯一吗?

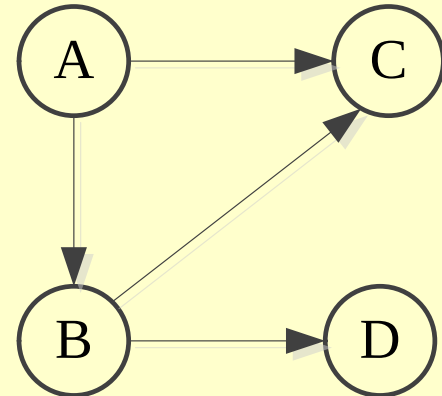


练习与作业：

1 已知无向图 G ，画出其分别用邻接矩阵与邻接表存储时的结构图。

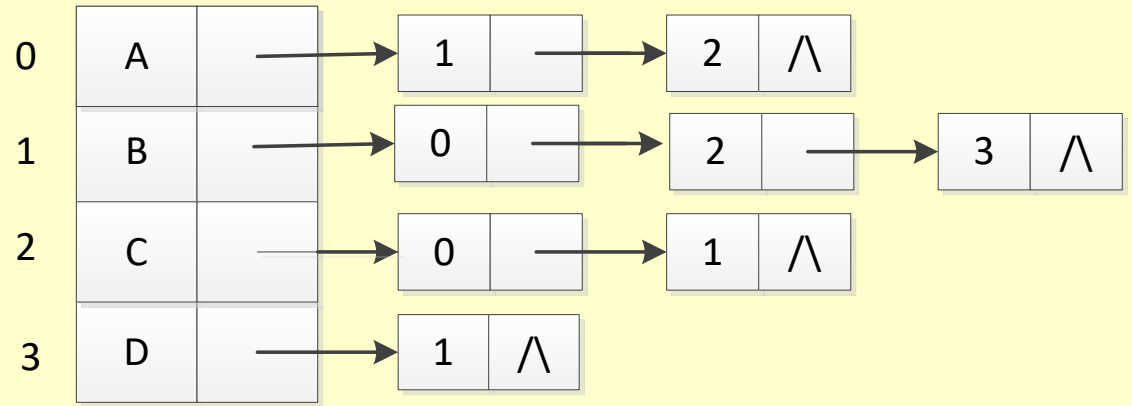


2 已知有向图 G ，画出其分别用邻接矩阵与邻接表存储时的结构图。

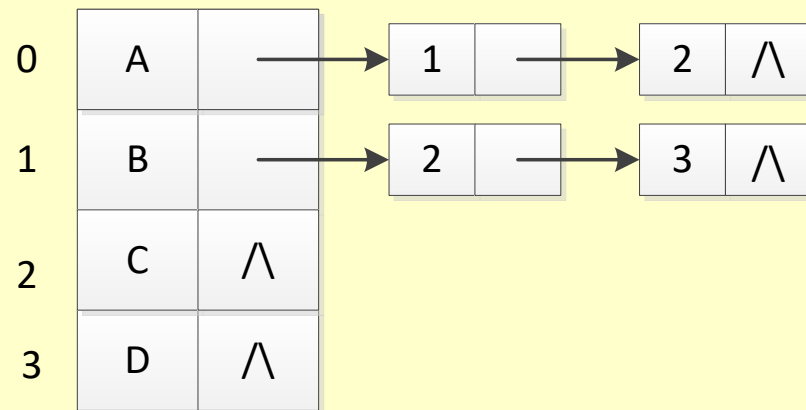


➤ 画出邻接多重表、十字链表表示

	A	B	C	D
A	0	1	1	0
B	1	1	0	1
C	1	1	0	0
D	0	1	0	0



	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	0
D	0	0	0	0

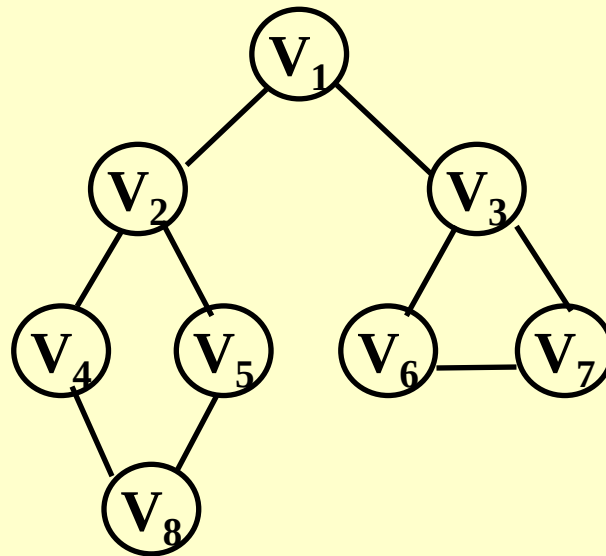


图结构遍历操作的复杂性:

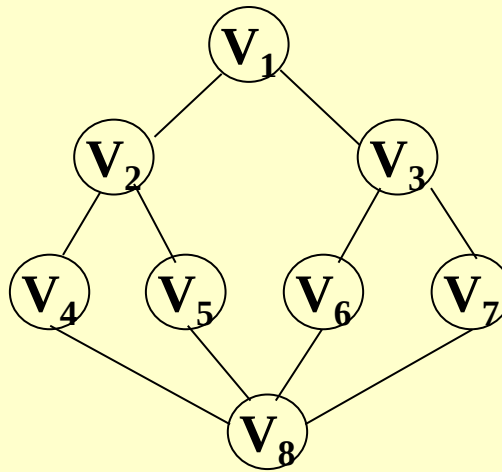
- 图中没有一个“自然”的首结点，图中任意一个顶点都可作为第一个被访问的结点。
- 在非连通图中，从一个顶点出发，只能够访问它所在的连通分量上的所有顶点，还需考虑如何选取下一个出发点以访问图中其余的连通分量。
- 在图结构中，如果有回路存在，那么一个顶点被访问之后，有可能沿回路又回到该顶点。
- 图中一个顶点可以和其它多个顶点相连，当这样的顶点访问过后，存在如何选取下一个要访问的顶点的问题。

一、深度优先遍历(DFS)

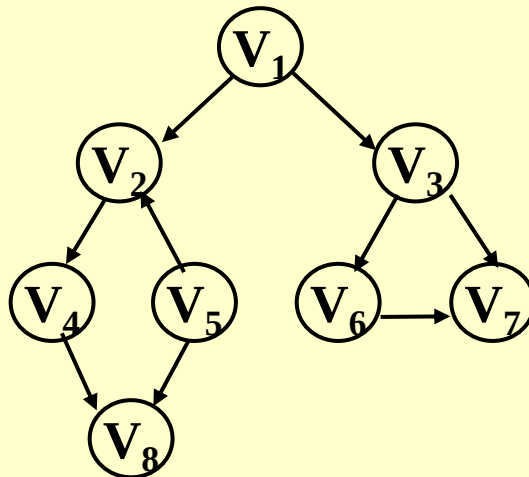
- 从图的某一顶点 V_0 出发，访问此顶点；
- 依次从 V_0 的未被访问的邻接点出发，深度优先遍历图，直至图中所有和 V_0 相通的顶点都被访问到；
- 若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点；
- 重复上述过程，直至图中所有顶点都被访问为止。



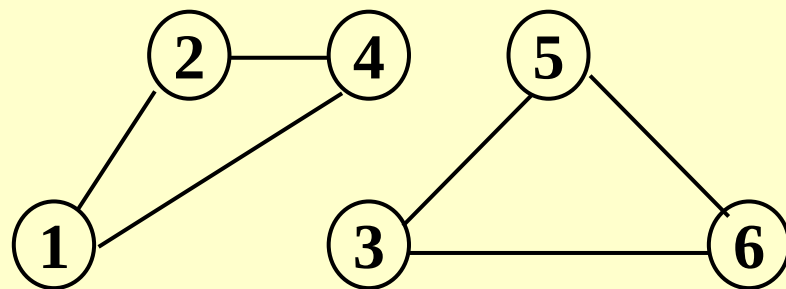
深度遍历： $V_1 \Rightarrow V_2 \Rightarrow V_4 \Rightarrow V_8 \Rightarrow V_5 \Rightarrow V_3 \Rightarrow V_6 \Rightarrow V_7$



深度遍历： $V_1 \Rightarrow V_2 \Rightarrow V_4 \Rightarrow V_8 \Rightarrow V_5 \Rightarrow V_6 \Rightarrow V_3 \Rightarrow V_7$



深度遍历： $V_1 \Rightarrow V_2 \Rightarrow V_4 \Rightarrow V_8 \Rightarrow V_3 \Rightarrow V_6 \Rightarrow V_7 \Rightarrow V_5$

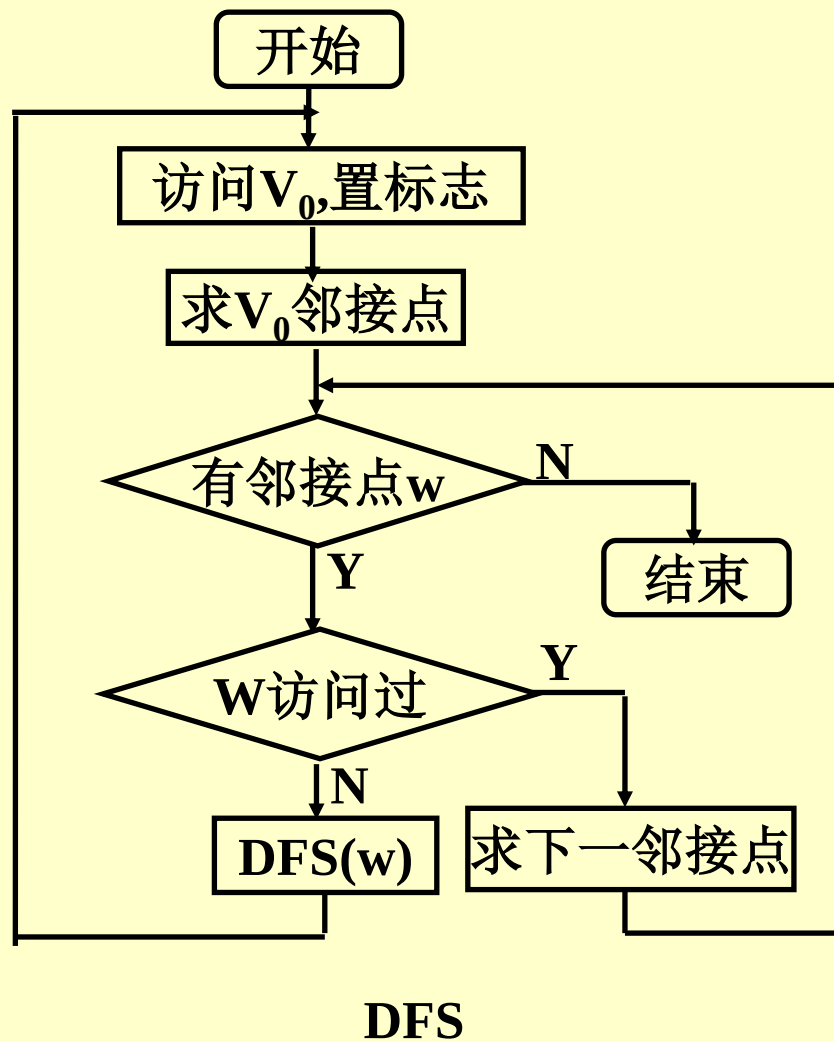
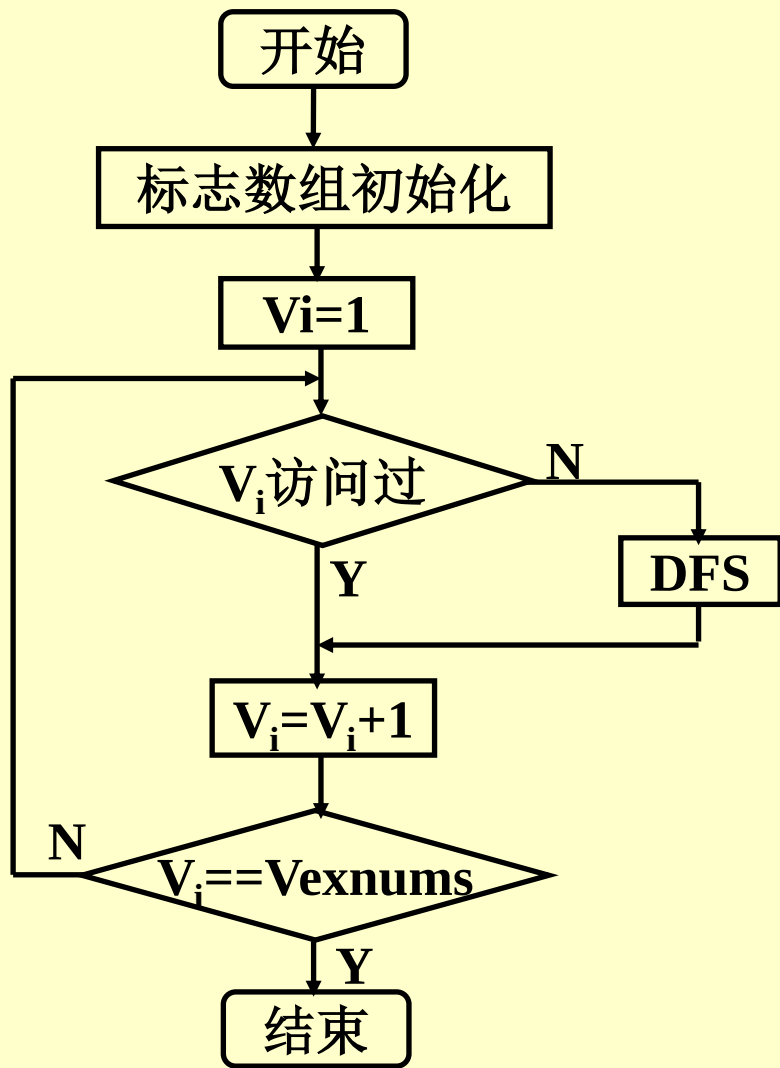


深度遍历: $V_1 \Rightarrow V_2 \Rightarrow V_4 \Rightarrow V_3 \Rightarrow V_6 \Rightarrow V_5$

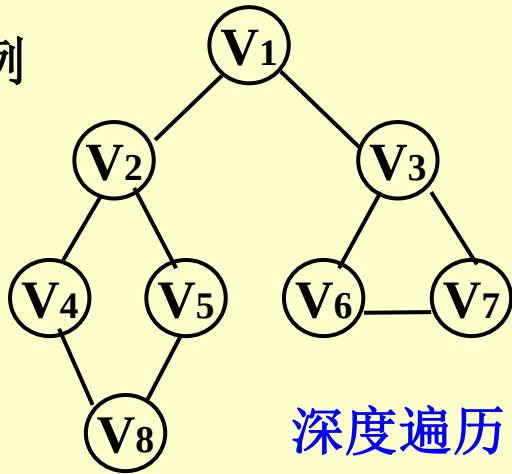
程序实现的关
键点?

需要使用辅助数组
`visited[0..n-1]`记录访问过的
节点。一般将`visted[i]`初值设
置为假,一旦`vi`被访问,便置
`visted[i]`为真。

★递归算法



例

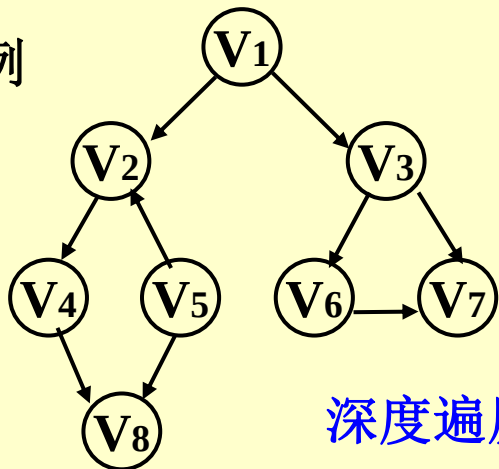


系统栈状态?

深度遍历: $V1 \Rightarrow V3 \Rightarrow V7 \Rightarrow V6 \Rightarrow V2 \Rightarrow V5 \Rightarrow V8 \Rightarrow V4$

	data	firstarc		adjvex	next		visited
0	V1	→	2	→	1	^	1
1	V2	→	4	→	3	→ 0	1
2	V3	→	6	→	5	→ 0	1
3	V4	→	7	→	1	^	1
4	V5	→	7	→	1	^	1
5	V6	→	6	→	2	^	1
6	V7	→	5	→	2	^	1
7	V8	→	4	→	3	^	1

例



深度遍历: $V1 \Rightarrow V3 \Rightarrow V7 \Rightarrow V6 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V5$

	vex	data	first	arc	adjvex	next
0	V1		—	→	2	1 ^
1	V2		—	→	3	^
2	V3		—	→	6	5 ^
3	V4		—	→	7	^
4	V5		—	→	7	1 ^
5	V6		—	→	6	^
6	V7		^			
7	V8		^			

//循环结合递归实现

void dfs(graph g, int k)

```
{
    int i;
    if(k == -1)
    {
        for(i=0;i<g.vexnum;i++)//处理非连通
            if(!visited[i])
                dfs(_____);
    }
    else
    {
        visited[k]=1;
        printf("%c ", g.vexs[k]);
        for(i=_____;i>=0;i=_____)
            if(!visited[i])
                dfs(_____);
    }
}
```

//返回k的第1个邻接点

//在邻接矩阵中查找相应行的第一个非无穷大的值

```
int firstvex_graph(graph g, int k) {
    int i;
    if(k>=0 && k<g.vexnum)
        for(i=0;i<g.vexnum;i++)
            if(g.arcs[k][i] != INFINITY)
                return i;
    return -1;
}
```

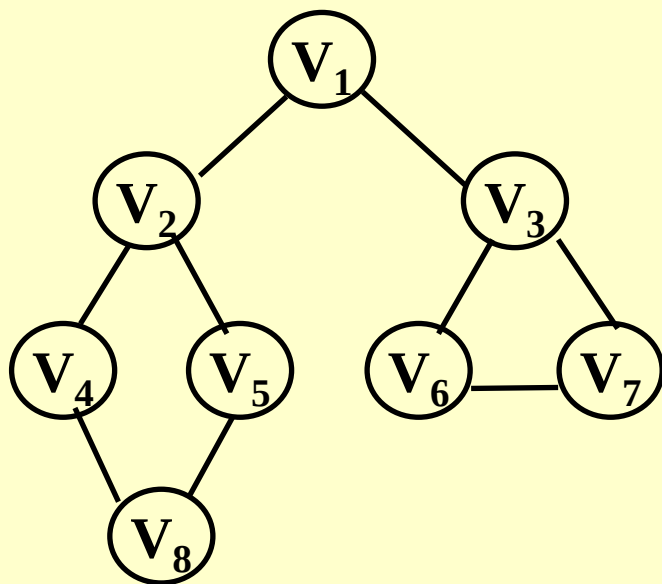
//返回i的第j个邻接点的下一个邻接点

//下一个非无穷大的值

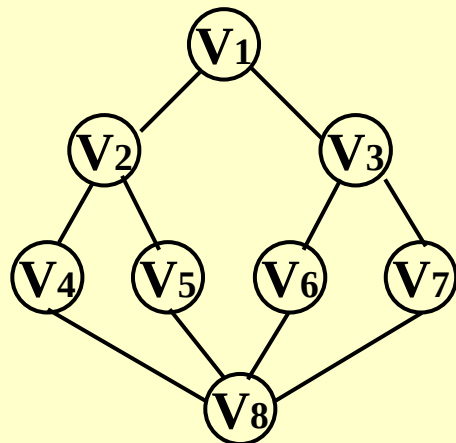
```
int nextvex_graph(graph g, int i, int j)
{   int k;
    if(i>=0 && i<g.vexnum && j>=0
        && j<g.vexnum)
        for(_____)
            if(_____)
                _____;
    return -1;
}
```

二、广度优先遍历(BFS)

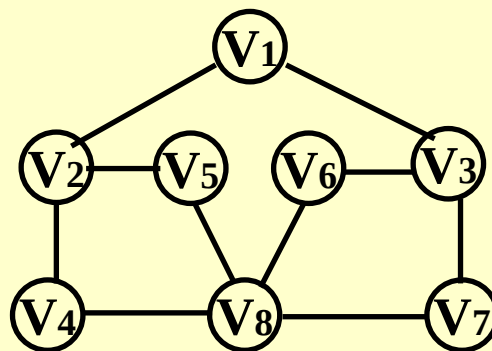
- 从图的某一顶点 V_0 出发，访问此顶点后，依次访问 V_0 的各个未曾访问过的邻接点；
- 然后分别从这些邻接点出发，广度优先遍历图，直至图中所有已被访问的顶点的邻接点都被访问到；
- 若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点，重复上述过程，直至图中所有顶点都被访问为止。



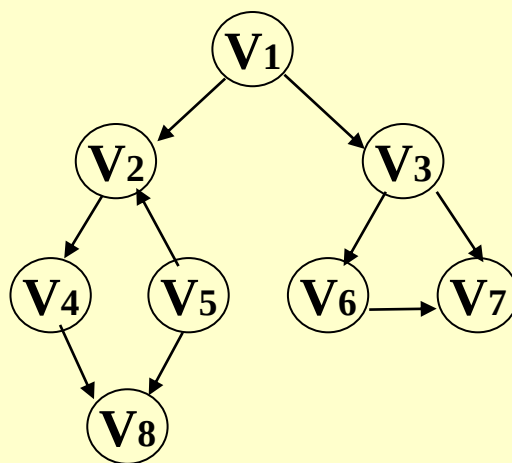
广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$



广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$

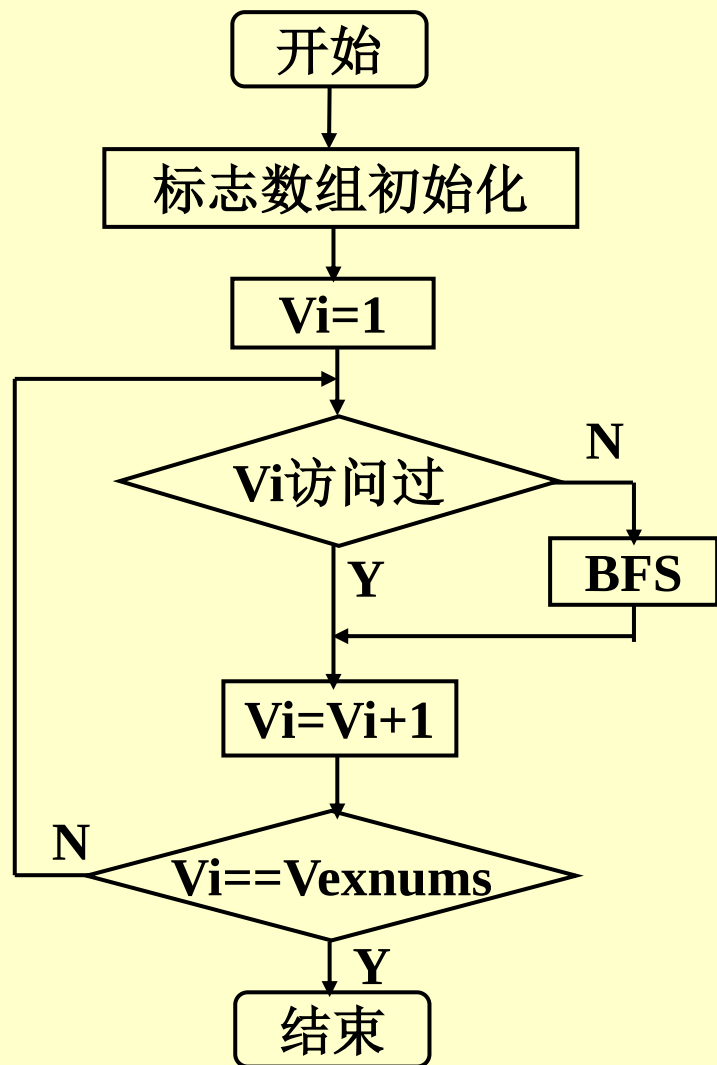


广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$

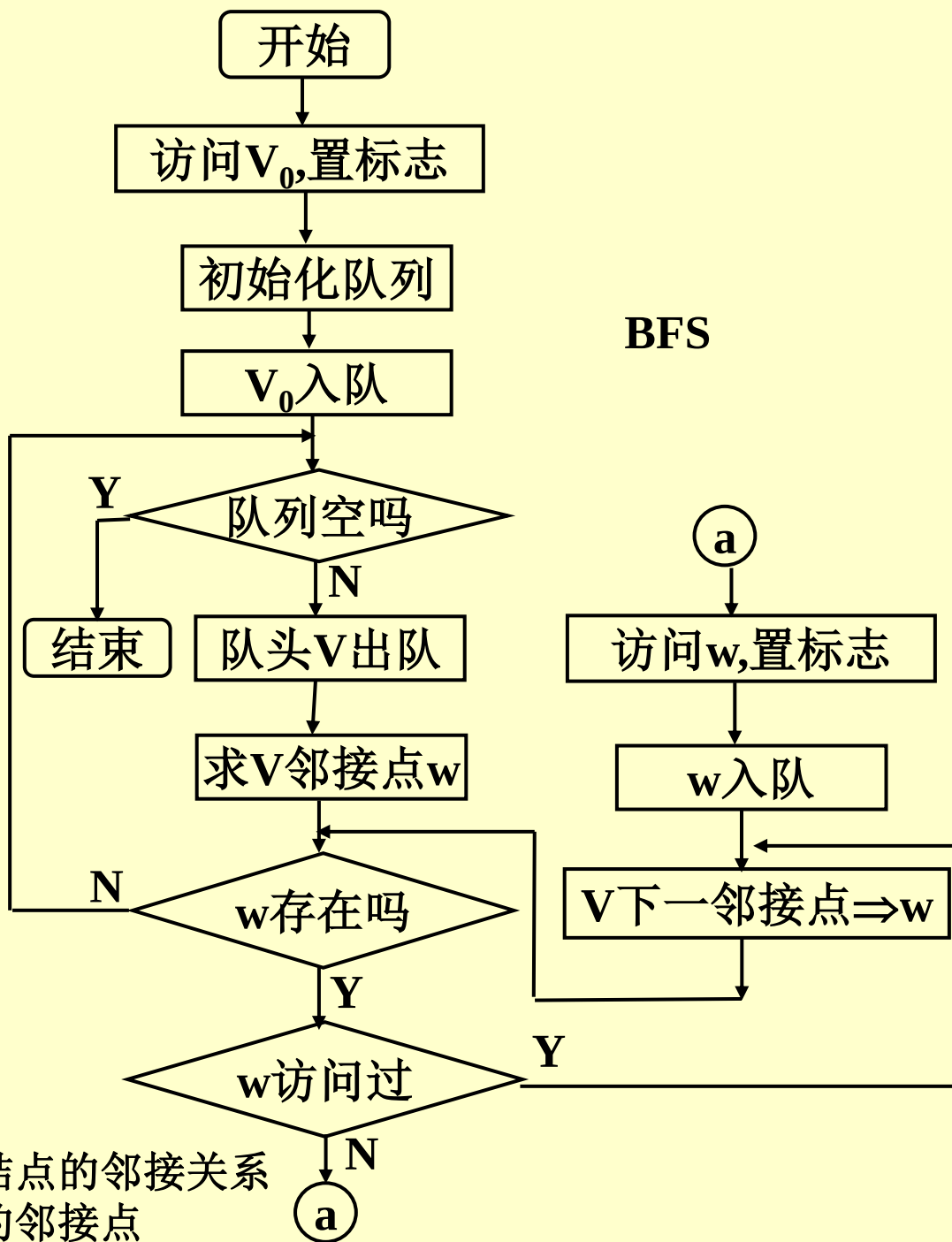


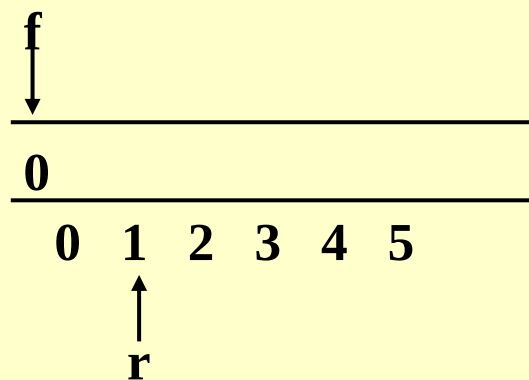
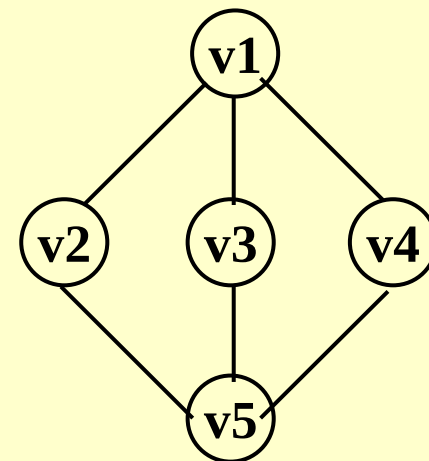
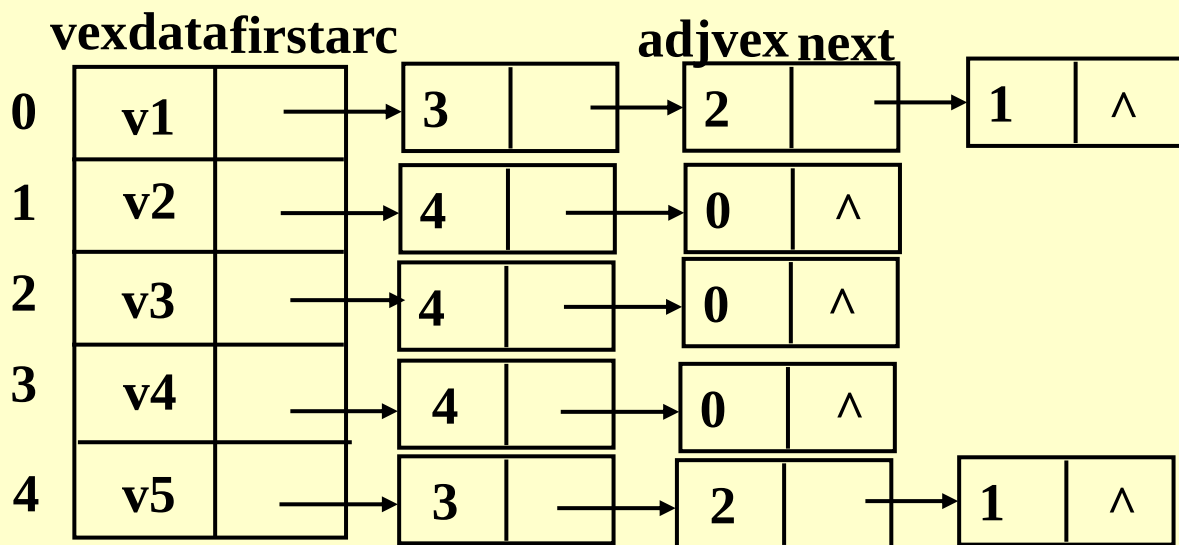
广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8 \Rightarrow V5$

广度优先遍历算法

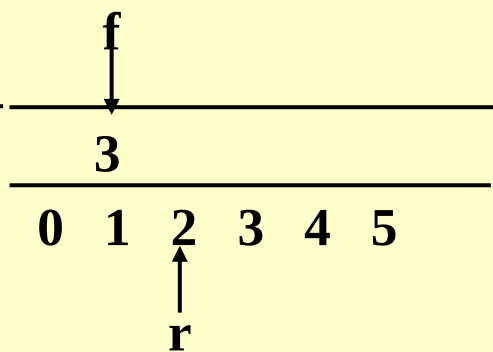


队列暂存当前结点的邻接关系
为了找邻接点的邻接点

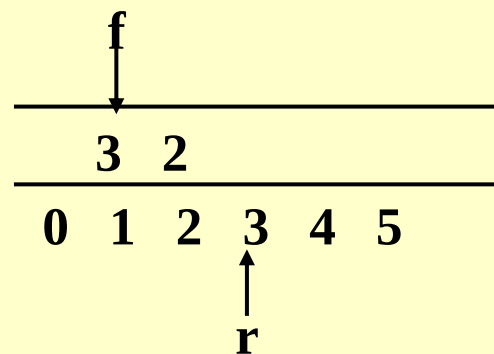




遍历序列: v1

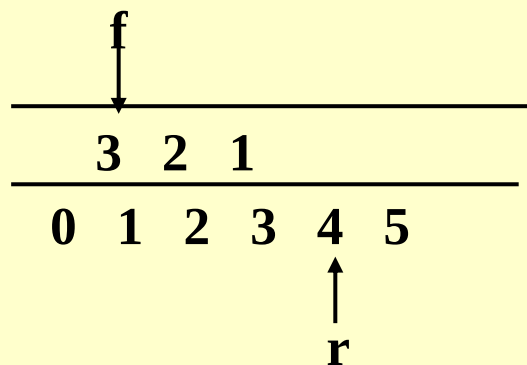
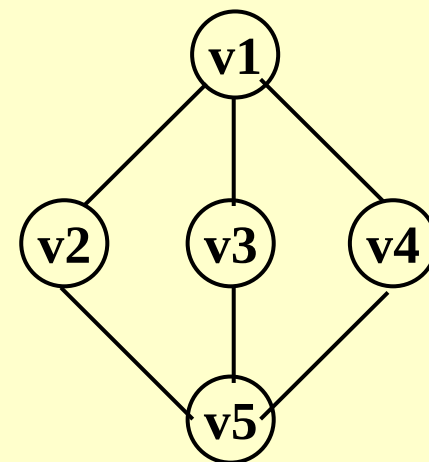
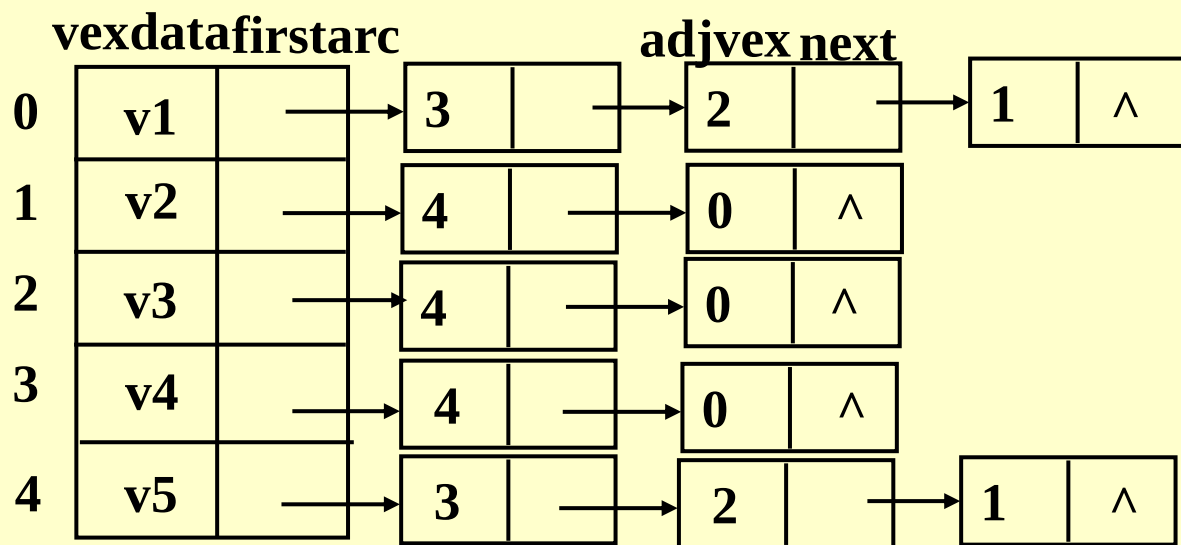


遍历序列: v1 v4

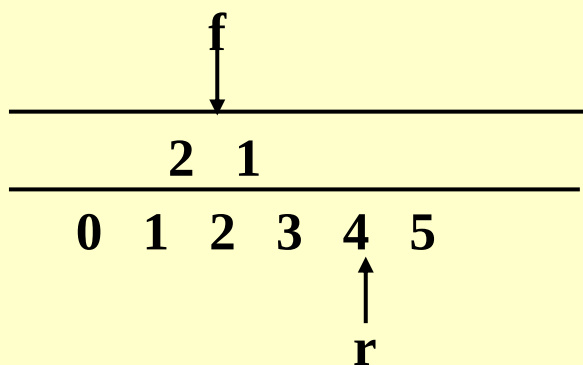


遍历序列: v1 v4 v3

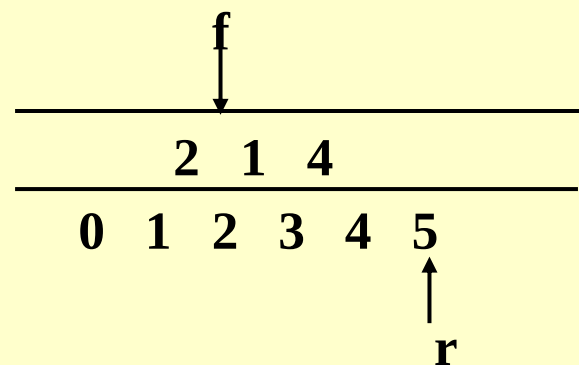
r用于计数



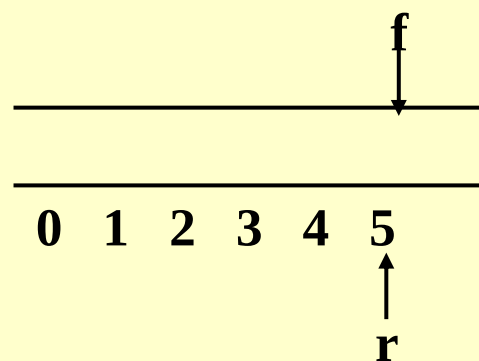
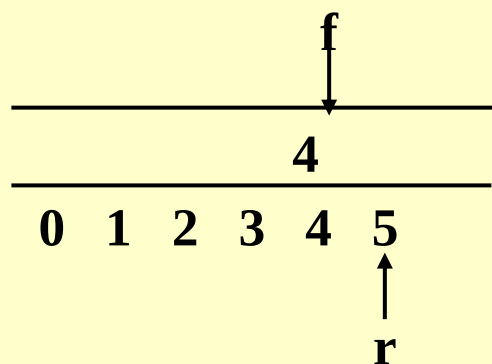
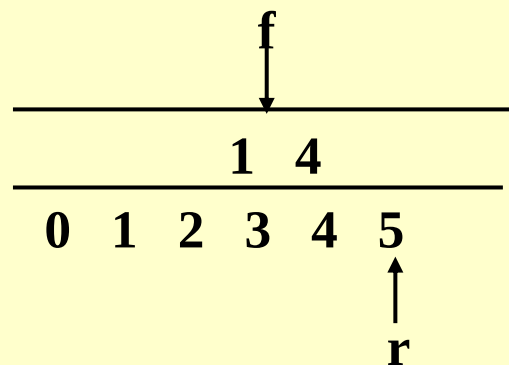
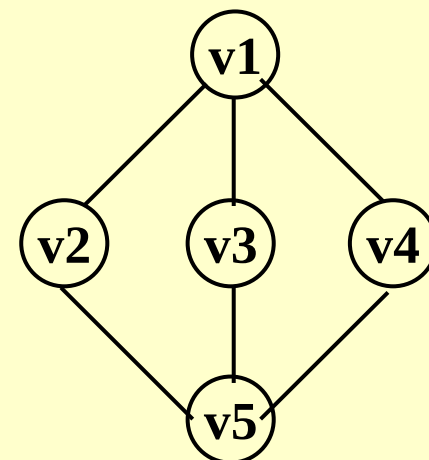
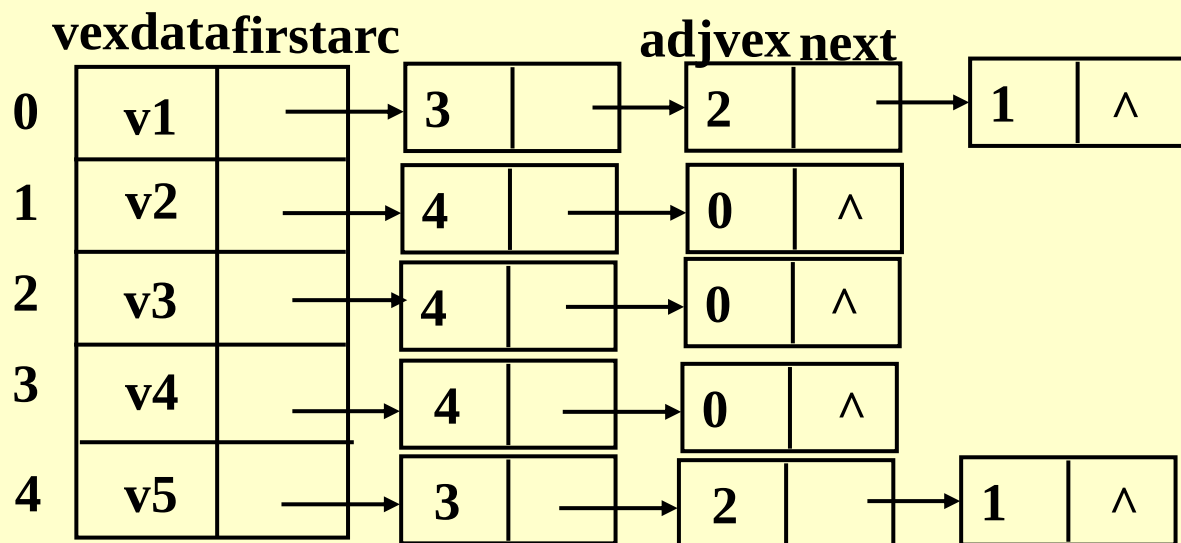
遍历序列: v1 v4 v3 v2



遍历序列: v1 v4 v3 v2



遍历序列: v1 v4 v3 v2 v5



遍历序列: v1 v4 v3 v2 v5

遍历序列: v1 v4 v3 v2 v5

遍历序列: v1 v4 v3 v2 v5

返回

//图的广度优先遍历算法，利用队列，非递归。

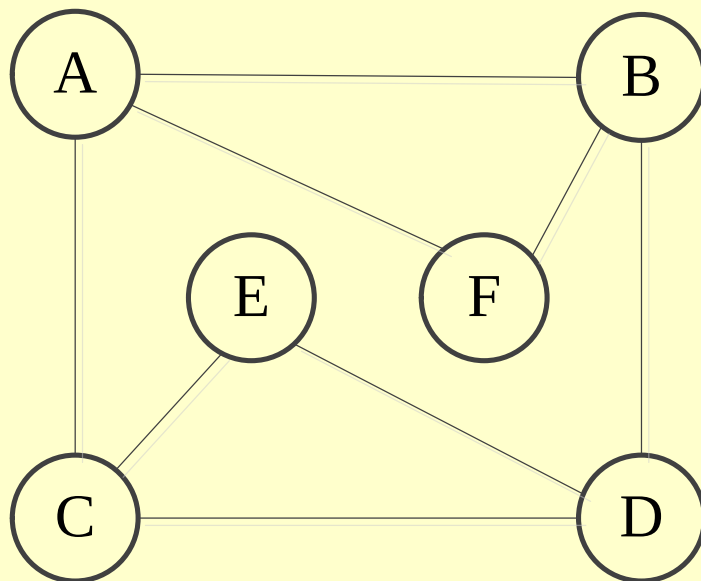
```
void bfs(graph g)
{   int i,j,k;
    queue qu;
    qu=init_queue();
    for(i=0;i<g.vexnum;i++)
        if(!visited[i])
        {
            visited[i] =1;
            printf("%c ", g.vexs[i]);
            _____; //i入队
            while(_____) //当队列qu非空
            {
                k=_____;//出队
                for(j=_____; j>=0;j=_____) //k的未被访问的邻接点入队
                    if(!visited[j])
                    {
                        visited[j]=1;
                        printf("%c ", g.vexs[j]);
                        _____; //j入队
                    }
            }
        }
}
```

```
//返回k的第1个邻接点
int firstvex_graph(graph g, int k) {
    int i;
    if(k>=0 && k<g.vexnum)
        for(i=0;i<g.vexnum;i++)
            if(g.arcs[k][i] != INFINITY)
                return i;
    return -1;
}
```

```
//返回j的第i个邻接点的下一个邻接点
int nextvex_graph(graph g, int i, int j)
{   int k;
    if(i>=0 && i<g.vexnum && j>=0
        && j<g.vexnum)
        for(_____)
            if(_____)
                _____;
    return -1;
}
```

练习与作业:

1. 给出深度优先遍历序列
 2. 给出广度优先遍历序列
- 假设存储结构按字母升序存储



Experiment 7-1 图的遍历算法邻接矩阵存储

Problems:

void CreatGraph_DN(ALGraph *G);

void CountVerNum_DN(ALGraph *G, int D[]);

void DFSTraverse (ALGraph *G);

void BFSTraverse (ALGraph *G);

- 19、一个有 n 个顶点的无向连通图，它所包含的连通分量个数为()。(中)
A、0 B、1 C、 n D、 $n+1$
20. 对于一个有向图，若一个顶点的入度为 k_1 、出度为 k_2 ，则对应邻接表中该顶点单链表中的结点数为()。(易)
A、 k_1 B、 k_2 C、 k_1-k_2 D、 k_1+k_2
- 21、对于一个有向图，若一个顶点的入度为 k_1 、出度为 k_2 ，则对应逆邻接表中该顶点单链表中的结点数为()。(中)
A、 k_1 B、 k_2 C、 k_1-k_2 D、 k_1+k_2
- 22、具有 n 个顶点的有向图最多有()条边。(中)
A、 n B、 $n(n-1)$ C、 $n(n+1)$ D、 n^2
- 23、 n 个顶点的强连通图至少有()条边。(中)
A、 n B、 $n-1$ C、 $n+1$ D、 $n(n-1)$
- 24、在一个具有 n 个顶点的有向图中，若所有顶点的出度之和为 s ，则所有顶点的入度之和为()。
A、 s B、 $s-1$ C、 $s+1$ D、 n
- 25、在一个无向图中，若两个顶点之间的路径长度为 k ，则该路径上的顶点数为()。(易)
A、 k B、 $k+1$ C、 $k+2$ D、 $2k$
- 30.带权有向图 G 用邻接矩阵 A 存储，则顶点 i 的入度等于 A 中() (中)
A、第 i 行非 ∞ 的元素之和 B、第 i 列非 ∞ 的元素之和
C、第 i 行非 ∞ 且非0的元素个数 D、第 i 列非 ∞ 且非0的元素个数
- 31.一个有 n 个顶点和 n 条边的无向图一定是()
A、连通的 B、不连通的 C、无环的 D、有环的
- 32.【2010统考真题】若无向图中含有7个顶点，要保证图在任何情况下都是连通的，则需要的边数最少是()
A、6 B、15 C、16 D、21
- 35.一个有28条边的非连通无向图至少有()个顶点。
A、7 B、8 C、9 D、10

40. 若图的邻接矩阵中主对角线上的元素皆为0，其余元素全为1，则可以断定该图一定（）。
- A、是无向图 B、是有向图
C、是完全图 D、不是带权图
41. 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中，零元素的个数为（）。
- A、 e B、 $2e$
C、 n^2-e D、 n^2-2e
42. 带权有向图 G 用邻接矩阵存储，则 v_i 的入度等于邻接矩阵中（）。
- A、第 i 行非 ∞ 的元素个数 B、第 i 列非 ∞ 的元素个数
C、第 i 行非 ∞ 且非0的元素个数 D、第 i 列非 ∞ 且非0的元素个数
43. 下列哪种图的邻接矩阵是对称矩阵？（）。
- A、有向网 B、无向网
C、AOV网 D、AOE网
44. 以下关于图的存储结构的叙述中，正确的是（）。
- A、一个图的邻接矩阵表示唯一，邻接表表示唯一
B、一个图的邻接矩阵表示唯一，邻接表表示不唯一
C、一个图的邻接矩阵表示不唯一，邻接表表示唯一
D、一个图的邻接矩阵表示不唯一，邻接表表示不唯一
45. 在有向图的邻接表存储结构中，顶点 v 在边表中出现的次数是（）。
- A、顶点 v 的度 B、顶点 v 的出度
C、顶点 v 的入度 D、依附于顶点 v 的边数

46. 对一个有 n 个顶点， e 条边的图采用邻接表示时，进行DFS遍历的时间复杂度为（）。

A、 $O(n)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(1)$

47. 对一个有 n 个顶点， e 条边的图采用邻接表表示时，进行DFS遍历的空间复杂度为（）。

A、 $O(n)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(1)$

48. 对一个有 n 个顶点， e 条边的图采用邻接表表示时，进行BFS遍历的时间复杂度为（）。

A、 $O(n)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(1)$

49. 对一个有 n 个顶点， e 条边的图采用邻接表表示时，进行BFS遍历的空间复杂度为（）。

A、 $O(n)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(1)$

50. 对一个有 n 个顶点， e 条边的图采用邻接矩阵表示时，进行DFS遍历的时间复杂度为（）。

A、 $O(n^2)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(e^2)$

51. 对一个有 n 个顶点， e 条边的图采用邻接矩阵表示时，进行BFS遍历的时间复杂度为（）。

A、 $O(n^2)$ B、 $O(e)$

C、 $O(n+e)$ D、 $O(e^2)$

52. 无向图 $G(V,E)$ ，其中 $V=\{a,b,c,d,e,f\}$ ， $E=\{(a,b), (a,e), (a,c), (b,e), (c,f), (f,d), (e,d)\}$ ，对该图从 a 开始进行深度优先遍历，得到的顶点序列正确的是（）。

A、 a,b,e,c,d,f B、 a,c,f,e,b,d

C、 a,e,b,c,f,d D、 a,e,d,f,c,b